

1. Pipes

POSIX Pipes

Example Code:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int fd[2];
    pid_t pid;
    char write_msg[20] = "Hello, POSIX Pipe!";
    char read_msg[20];

    // Create a pipe
    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {
        // Child process
        close(fd[1]); // Close write end
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child read: %s\n", read_msg);
        close(fd[0]);
    } else {
        // Parent process
        close(fd[0]); // Close read end
        write(fd[1], write_msg, sizeof(write_msg));
        printf("Parent wrote: %s\n", write_msg);
        close(fd[1]);
    }

    return 0;
}
```

System V Pipes

System V Unix pipes work in the same way as POSIX pipes since both are based on the `pipe` function.

2. Message Queues

POSIX Message Queues

Example Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mqueue.h>
#include <fcntl.h>
#include <sys/stat.h>

int main() {
    mqd_t mq;
    struct mq_attr attr;
    char buffer[128];

    attr.mq_flags = 0;
    attr.mq_maxmsg = 10;
    attr.mq_msgsize = 128;
    attr.mq_curmsgs = 0;

    mq = mq_open("/posix_mq", O_CREAT | O_WRONLY, 0644, &attr);
    if (mq == (mqd_t)-1) {
        perror("mq_open");
        exit(1);
    }

    strcpy(buffer, "Hello from POSIX message queue");
    if (mq_send(mq, buffer, 128, 0) == -1) {
        perror("mq_send");
        exit(1);
    }

    mq_close(mq);
    return 0;
}

```

Receiver:

```

#include <stdio.h>
#include <stdlib.h>
#include <mqueue.h>
#include <fcntl.h>

int main() {
    mqd_t mq;
    char buffer[128];

    mq = mq_open("/posix_mq", O_RDONLY);
    if (mq == (mqd_t)-1) {
        perror("mq_open");
        exit(1);
    }

    if (mq_receive(mq, buffer, 128, NULL) == -1) {
        perror("mq_receive");
        exit(1);
    }
}

```

```
    printf("Received message: %s\n", buffer);
    mq_close(mq);
    mq_unlink("/posix_mq");
    return 0;
}
```

System V Message Queues

Example Code:

Sender:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/msg.h>

struct message {
    long mtype;
    char mtext[128];
};

int main() {
    key_t key;
    int msgid;
    struct message msg;

    key = ftok("sysv_msg_send.c", 'B');
    msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget");
        exit(1);
    }

    msg.mtype = 1;
    strcpy(msg.mtext, "Hello from System V message queue");

    if (msgsnd(msgid, &msg, sizeof(msg.mtext), 0) == -1) {
        perror("msgsnd");
        exit(1);
    }

    return 0;
}
```

Receiver:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/msg.h>

struct message {
    long mtype;
    char mtext[128];
};
```

```

int main() {
    key_t key;
    int msgid;
    struct message msg;

    key = ftok("sysv_msg_send.c", 'B');
    msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget");
        exit(1);
    }

    if (msgrcv(msgid, &msg, sizeof(msg.mtext), 1, 0) == -1) {
        perror("msgrcv");
        exit(1);
    }

    printf("Received message: %s\n", msg.mtext);

    msgctl(msgid, IPC_RMID, NULL);

    return 0;
}

```

3. Shared Memory

POSIX Shared Memory

Example Code:

Writer:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <string.h>

int main() {
    int shm_fd;
    void *ptr;

    shm_fd = shm_open("/posix_shm", O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, 4096);
    ptr = mmap(0, 4096, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    sprintf(ptr, "Hello from POSIX shared memory");
    return 0;
}

```

Reader:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

```

```

#include <sys/mman.h>
#include <unistd.h>

int main() {
    int shm_fd;
    void *ptr;

    shm_fd = shm_open("/posix_shm", O_RDONLY, 0666);
    ptr = mmap(0, 4096, PROT_READ, MAP_SHARED, shm_fd, 0);

    printf("%s\n", (char *)ptr);
    shm_unlink("/posix_shm");
    return 0;
}

```

System V Shared Memory

Example Code:

Writer:

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <string.h>

int main() {
    int segment_id;
    char *shared_memory;

    segment_id = shmget(IPC_PRIVATE, 4096, IPC_CREAT | IPC_EXCL | S_IRUSR | S_IWUSR);
    shared_memory = (char *)shmat(segment_id, NULL, 0);

    sprintf(shared_memory, "Hello from System V shared memory");

    return 0;
}

```

Reader:

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main() {
    int segment_id;
    char *shared_memory;

    segment_id = shmget(IPC_PRIVATE, 4096, S_IRUSR | S_IWUSR);
    shared_memory = (char *)shmat(segment_id, NULL, 0);

    printf("%s\n", shared_memory);

    shmctl(segment_id, IPC_RMID, NULL);
}

```

```
    return 0;
}
```

4. Semaphores

POSIX Semaphores

Example Code:

Semaphore Initialization and Usage:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

sem_t semaphore;

void* thread_func(void* arg) {
    sem_wait(&semaphore);
    printf("Thread %d entered critical section\n", *((int *)arg));
    sleep(2);
    printf("Thread %d leaving critical section\n", *((int *)arg));
    sem_post(&semaphore);
    return NULL;
}

int main() {
    pthread_t thread1, thread2;
    int id1 = 1, id2 = 2;

    sem_init(&semaphore, 0, 1);

    pthread_create(&thread1, NULL, thread_func, &id1);
    pthread_create(&thread2, NULL, thread_func, &id2);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    sem_destroy(&semaphore);
    return 0;
}
```

System V Semaphores

Example Code:

Semaphore Initialization and Usage:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <unistd.h>
```

```

union semun {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

void P(int semid) {
    struct sembuf p = {0, -1, SEM_UNDO}; // Decrease semaphore
    semop(semid, &p, 1);
}

void V(int semid) {
    struct sembuf v = {0, 1, SEM_UNDO}; // Increase semaphore
    semop(semid, &v, 1);
}

int main() {
    key_t key;
    int semid;
    union semun arg;

    // Generate a unique key for semaphore
    key = ftok("sysv_sem.c", 'E');
    semid = semget(key, 1, 0666 | IPC_CREAT); // Create semaphore set

    arg.val = 1; // Initialize semaphore value to 1
    semctl(semid, 0, SETVAL, arg); // Set semaphore value

    if (fork() == 0) {
        // Child process
        P(semid); // Lock semaphore (Enter critical section)
        printf("Child process: Entered critical section.\n");
        sleep(2); // Simulate some work
        printf("Child process: Leaving critical section.\n");
        V(semid); // Unlock semaphore (Leave critical section)
    } else {
        // Parent process
        P(semid); // Lock semaphore (Enter critical section)
        printf("Parent process: Entered critical section.\n");
        sleep(2); // Simulate some work
        printf("Parent process: Leaving critical section.\n");
        V(semid); // Unlock semaphore (Leave critical section)
    }

    // Cleanup
    if (fork() != 0) { // Only parent removes the semaphore
        semctl(semid, 0, IPC_RMID, arg); // Remove semaphore set
    }

    return 0;
}

```

Explanation of the Semaphore Example

1. Initialization:

- A **System V semaphore** is created using `semget` with a unique key generated by `ftok`.
- Its initial value is set using `semctl` with the `SETVAL` command.

2. P() and V() Operations:

- The `P()` function (proberen, or "test" in Dutch) decreases the semaphore count and enters the critical section when it is non-negative.
- The `V()` function (verhogen, or "increment" in Dutch) increases the semaphore count, signaling that the critical section is released.

3. Critical Section:

- Both parent and child processes attempt to access the critical section.
- The semaphore ensures that only one process is allowed into the critical section at any given time.

4. Cleanup:

- After usage, the semaphore is removed using `semctl` with the `IPC_RMID` command to free resources.

Comparing the Two Standards

Feature	POSIX Semaphores	System V Semaphores
API	Functions like <code>sem_init</code> , <code>sem_wait</code> , and <code>sem_post</code>	Uses <code>semget</code> , <code>semop</code> , and <code>semctl</code>
Ease of Use	Easier and more modern	Slightly more complex due to legacy syntax
Namespace	Named or unnamed semaphores	Identified by keys generated using <code>ftok</code>
Portability	POSIX-compliant systems	Limited to System V or Unix-based systems

Summary of Key Differences in IPC Mechanisms

Here's a brief recap:

- **Pipes:** Both POSIX and System V use unnamed pipes. However, POSIX also allows named pipes (FIFOs).
- **Message Queues:** POSIX message queues are easier to use with named references, while System V queues rely on `ftok` keys.

- **Shared Memory:** POSIX shared memory uses file descriptors and names (e.g., `/shm_name`), while System V uses `ftok` keys and `shmid`.
- **Semaphores:** POSIX semaphores are simpler and support thread synchronization, while System V semaphores are system-wide and more complex.